

Rompiendo el candado: de la explotación al parche

Arquitectura segura, vulnerabilidades web y
defensa estructural

Jonathan Zenteno G.
Ingeniero Informático y Facilitador REUF SENCE



Jonathan Zenteno G.

Ingeniero Informático, Coordinador Informático
DAEM Linares / SLEP Los Álamos.

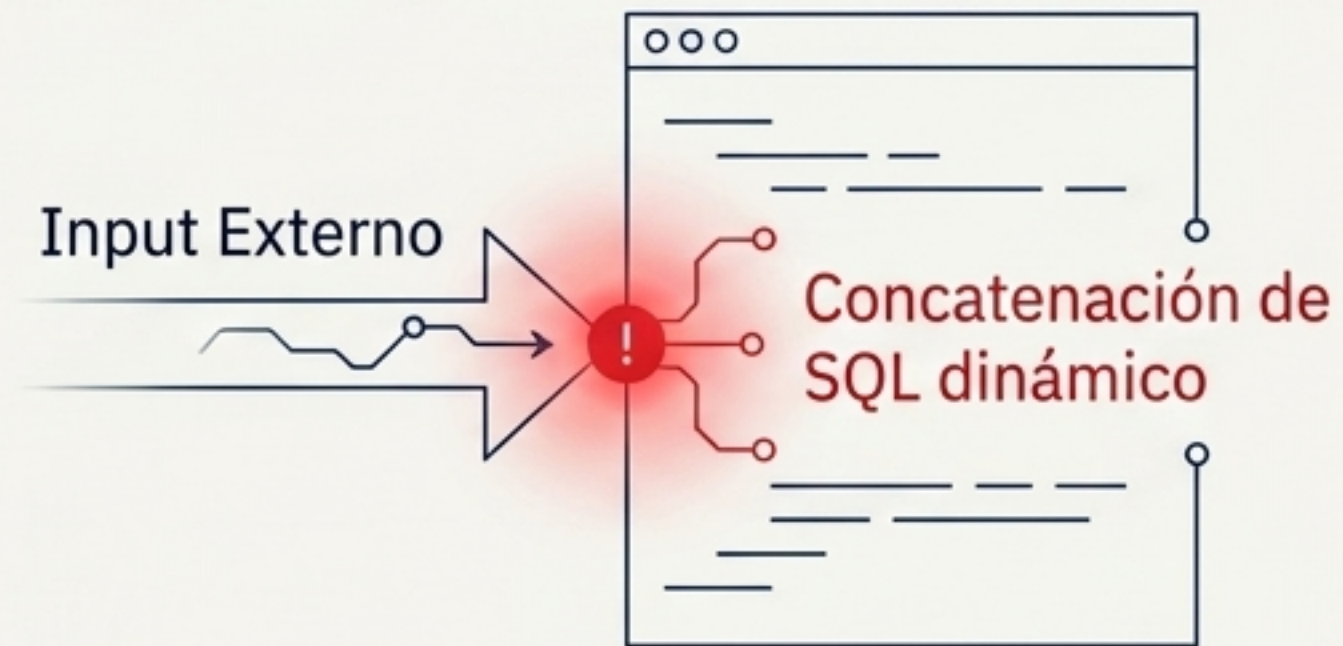
Ciberseguridad, Gestión de Infraestructura
(SIGE/SAE), Formación de Formadores
(SENCE/ADA).



Mantra: Transformar la mentalidad técnica de
buscar trucos a diseñar arquitecturas seguras
por defecto.

El origen de la falla estructural: Datos vs. Código

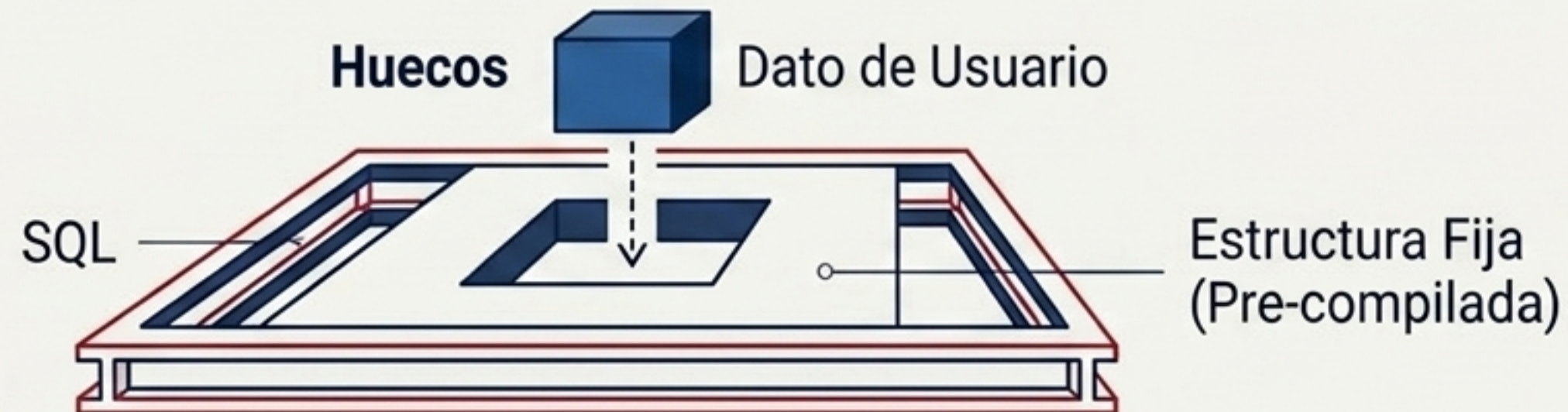
El problema no son los caracteres especiales.
El problema es la pérdida de separación de intenciones.



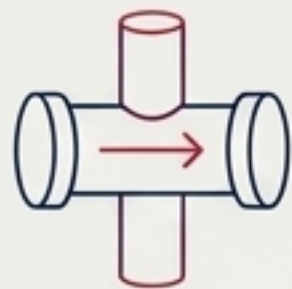
La vulnerabilidad nace exactamente en el punto donde la aplicación mezcla la estructura de la consulta (lo que el programador quiere ejecutar) con los datos del usuario (lo que el usuario quiere buscar), permitiendo que el dato modifique la instrucción.

El falso escudo: Validación de formato vs. Defensa estructural

Validación	Parametrización
Responde a la pregunta: ¿Es el formato correcto? (ej. ¿Es un número?).	Responde a la pregunta: ¿Puede el dato convertirse en instrucción?
Reduce basura y mejora la calidad de los datos.	El motor compila la estructura antes de insertar el dato.
NO previene inyecciones estructurales.	El dato nunca se evalúa como código.



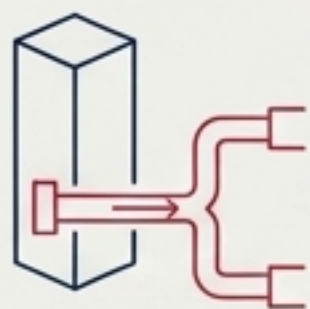
Taxonomía del ataque y la evaluación del daño



In-band
(mismo canal)



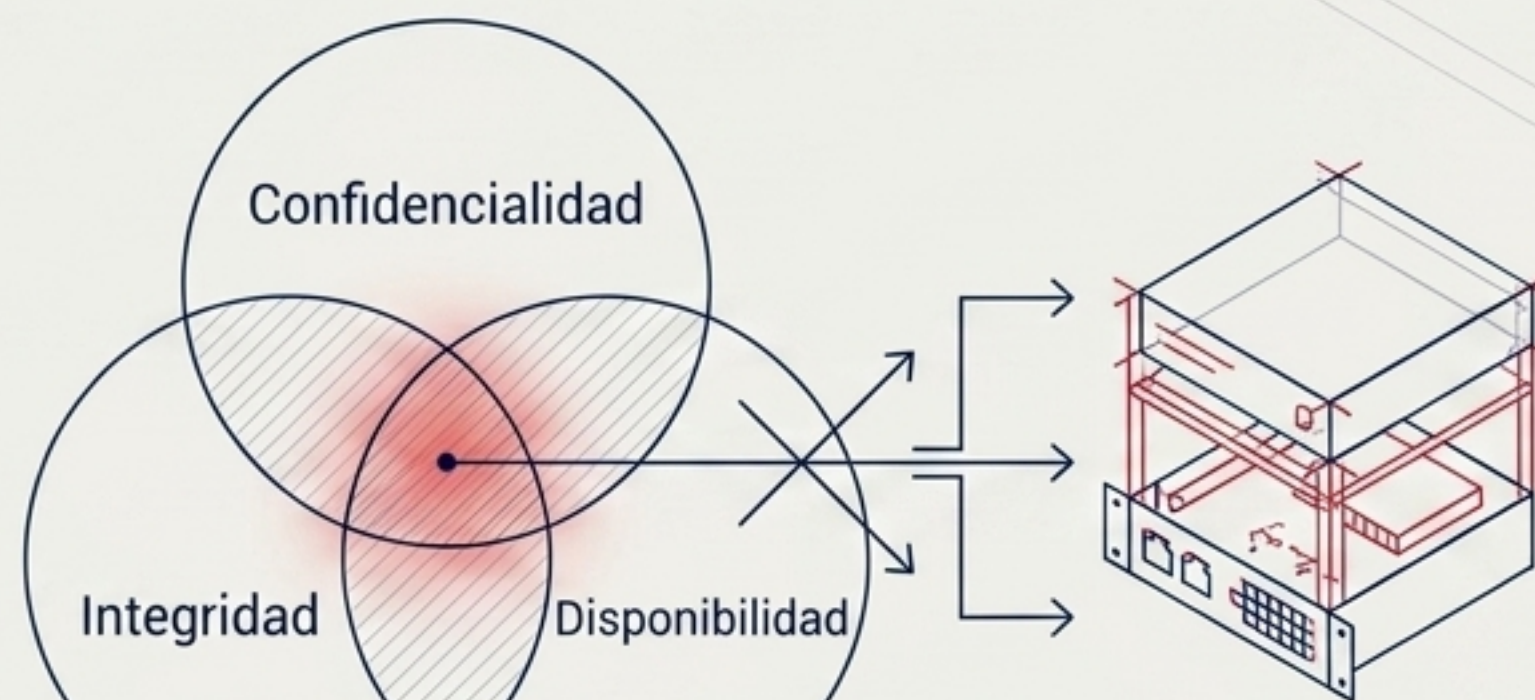
Blind/Inferencial
(deducción por
comportamiento)



Out-of-band
(canal alternativo)

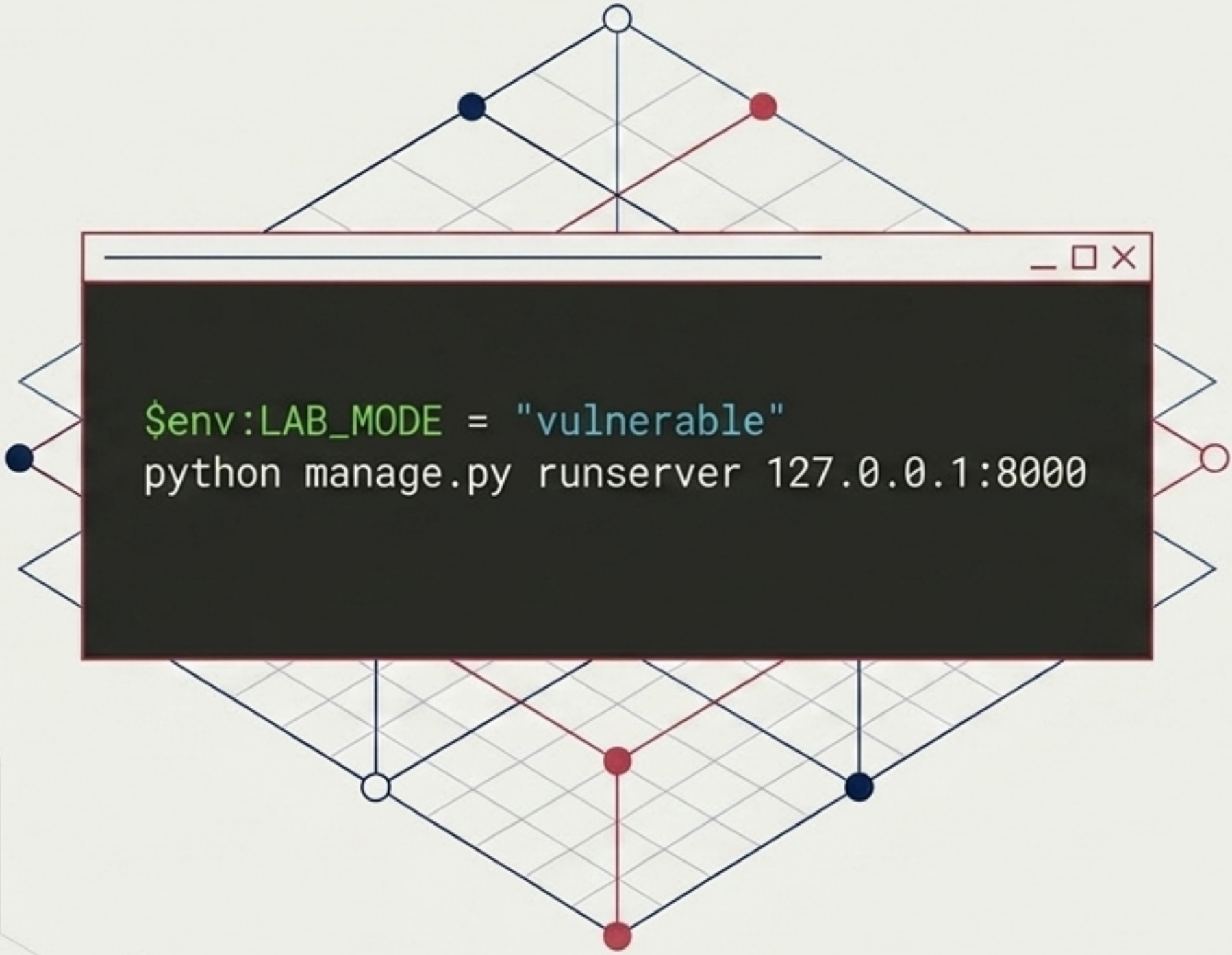


Error-based
(fuga de información)



**Una vulnerabilidad no es igual a su impacto.
El daño a la tríada C.I.D. no lo define el
payload, sino los permisos y la arquitectura
de la base de datos comprometida.**

Entrando al modo táctico: Laboratorio CTF



```
$env:LAB_MODE = "vulnerable"  
python manage.py runserver 127.0.0.1:8000
```

Reglas del juego defensivo



Este entorno es para observar la teoría en la práctica y aplicar mitigaciones (ORM, parametrización).



Todo ataque sin permiso explícito a sistemas externos constituye un delito (Ley 21.459).

Reto 1: Bypass de Autenticación

Ataque: Comment Injection

Username

Password

```
SELECT id, username FROM auth_user  
WHERE username = 'admin' \u002d\u002d  
AND password = 'x'
```

El payload `admin' \u002d\u002d` apaga el resto de la validación.

Defensa: ORM Seguro

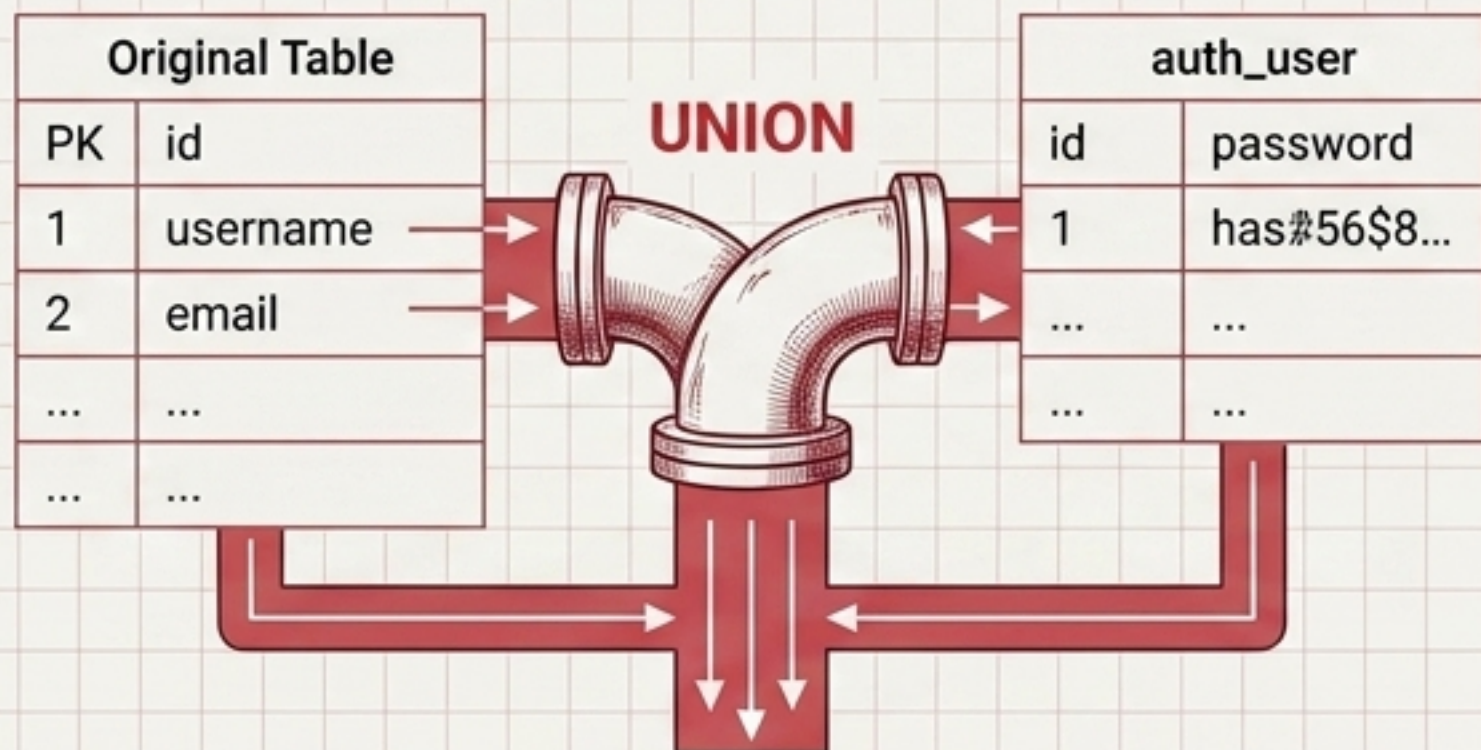
`authenticate()`

Uso del ORM de Django.
Emplea consultas parametrizadas por defecto y compara contra hashes, impidiendo la concatenación del input en la sentencia SQL.

Reto 2: Extracción con UNION y Exfiltración de Hashes

Explotación

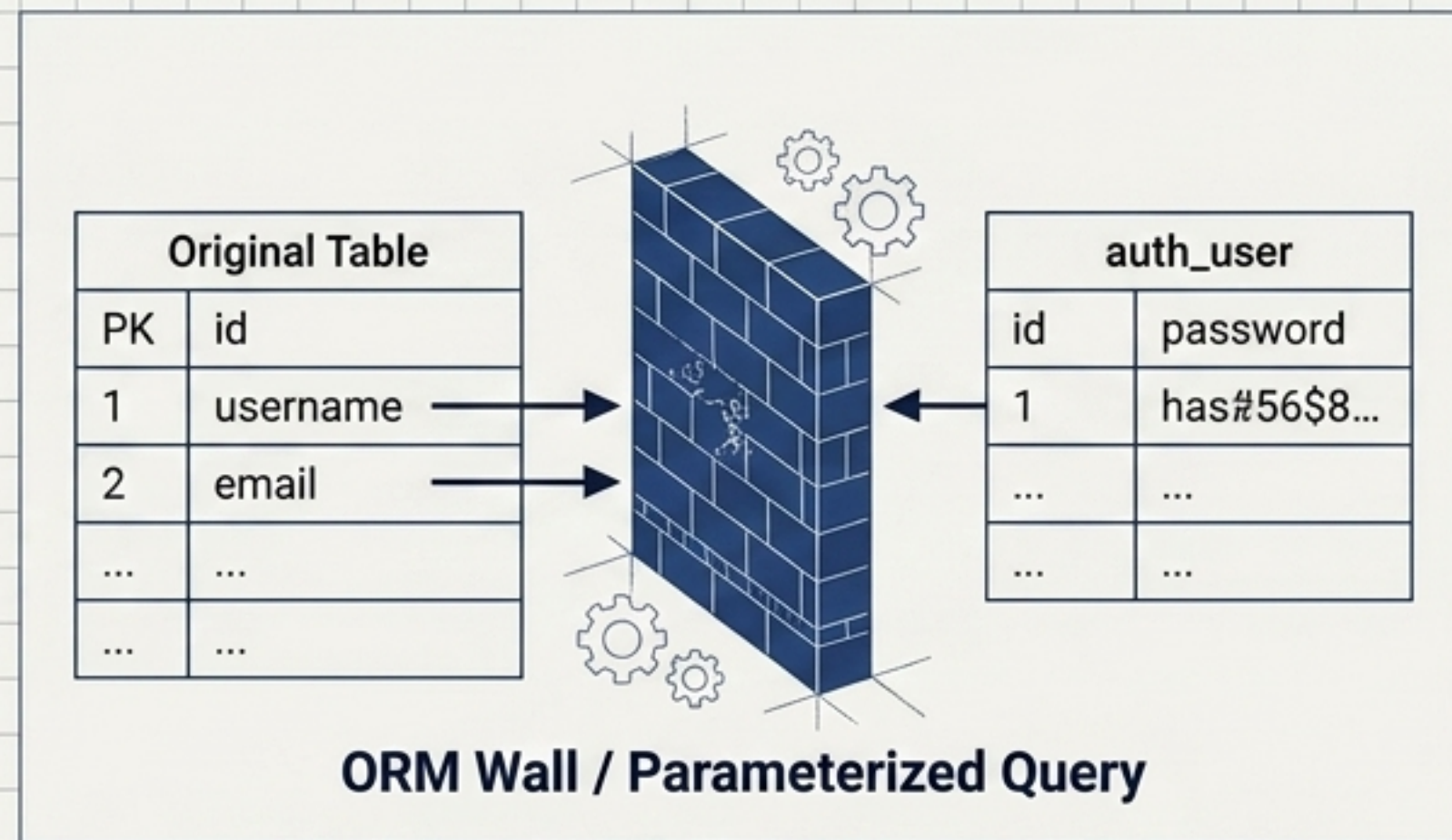
```
' UNION SELECT id, password FROM auth_user --
```



Resultado: pbkdf2_sha256\$870000\$...

Por qué vemos un hash PBKDF2 y no la clave plana:
Defensa en profundidad en almacenamiento de contraseñas.

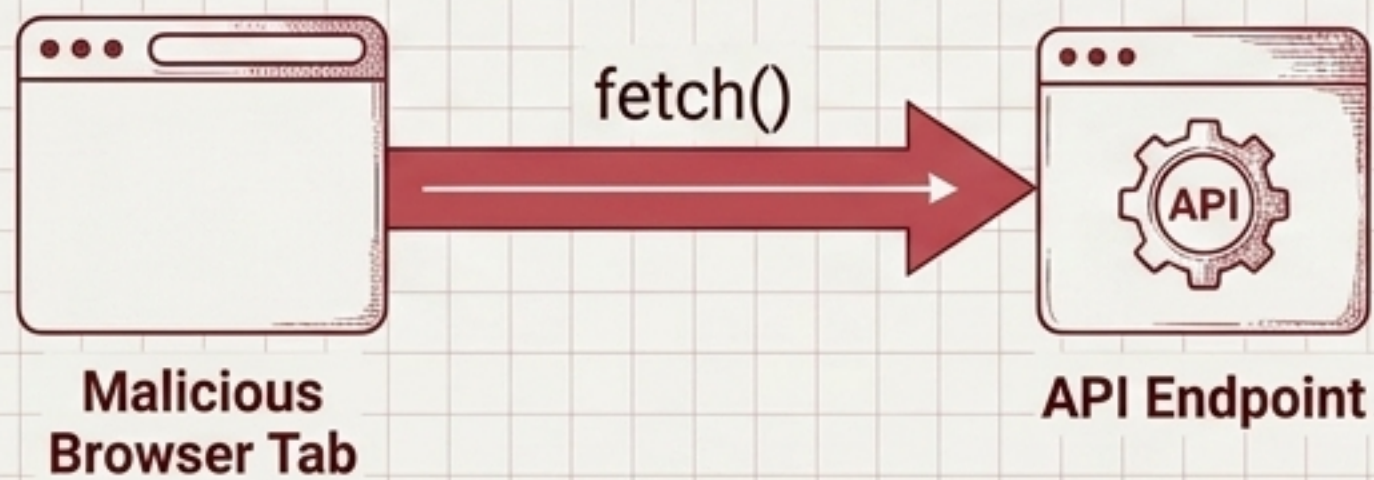
El Parche



El ORM evalúa el input de usuario de forma estrictamente literal, destruyendo cualquier intento de alterar la estructura del SELECT original.

Reto 3: Transferencias Fantasma (CSRF) y la Confianza Implícita

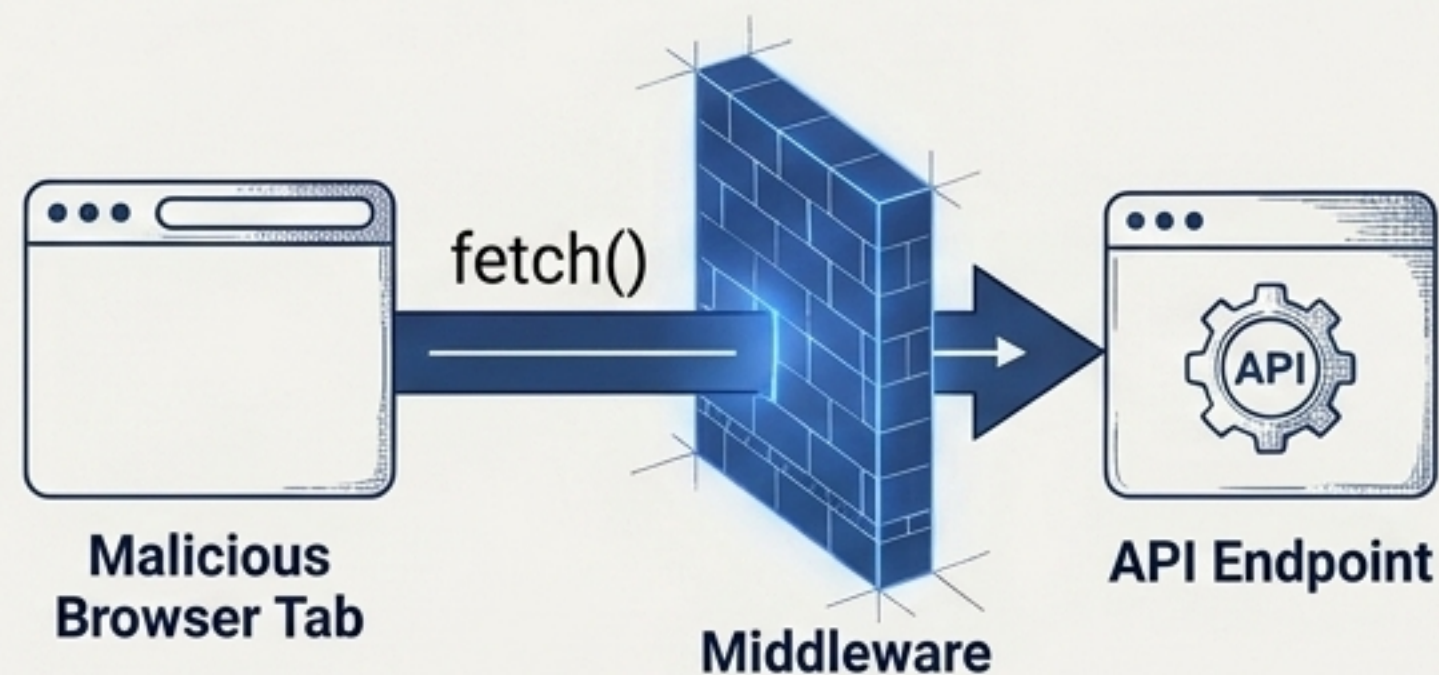
Explotación



`@csrf_exempt`

Ejecución de estado sin validación de origen. El peligro del decorador `@csrf_exempt` en Django, permitiendo peticiones cruzadas sin el token de validación.

El Parche

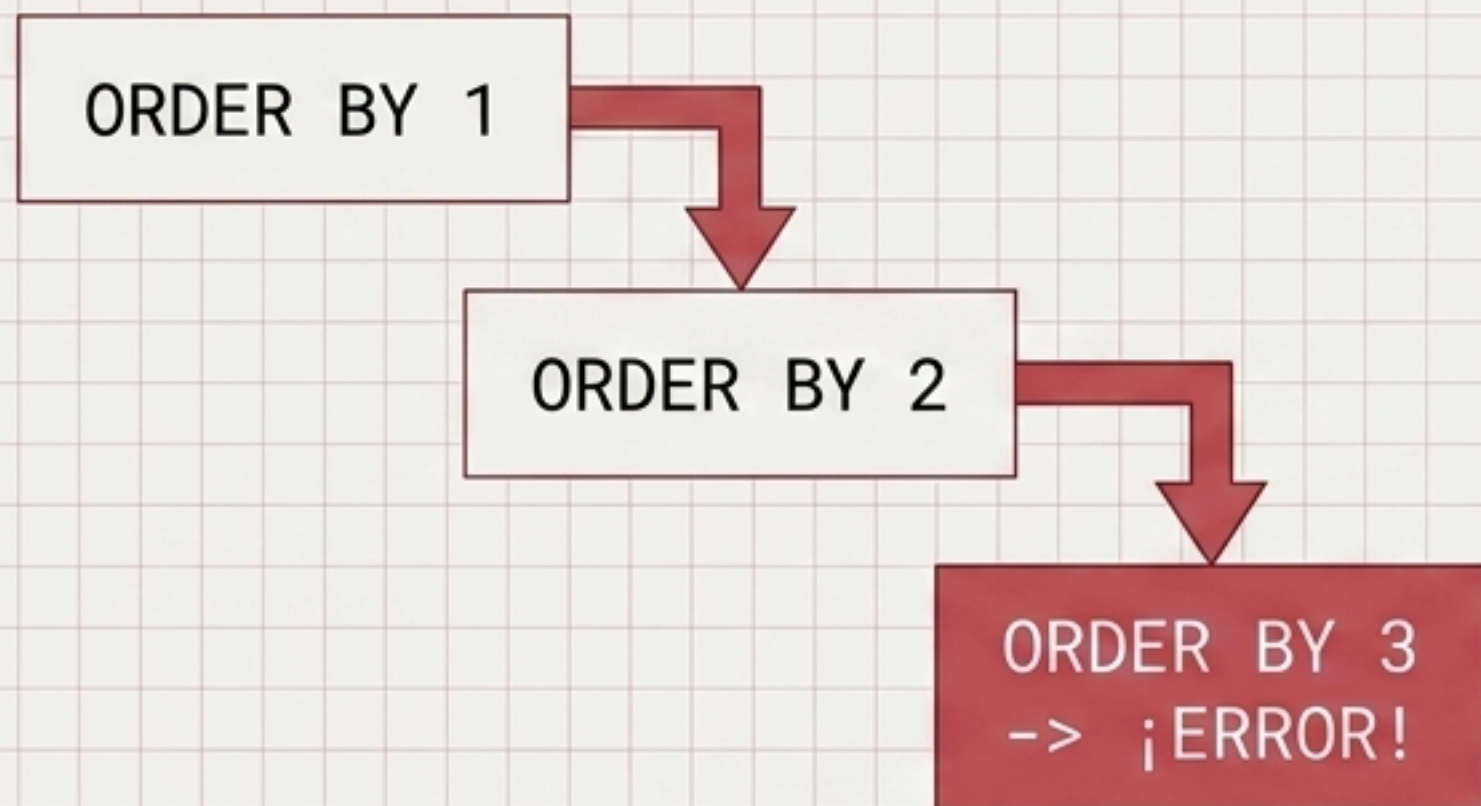


`CsrfViewMiddleware`

Django protege por defecto actuando como guardián en la frontera; eliminar exenciones injustificadas restaura la seguridad.

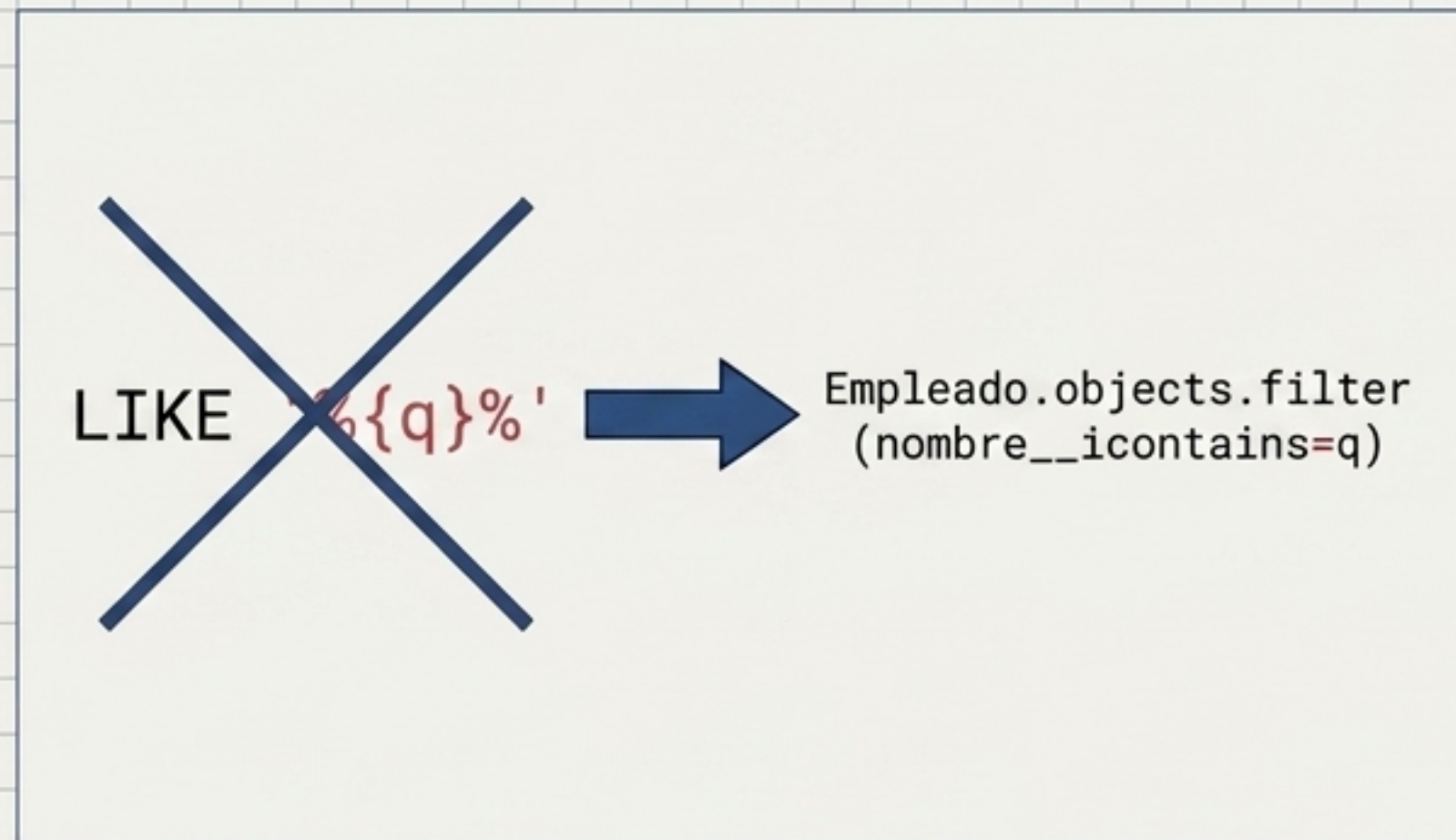
Reto 4: Mapeo a Ciegas y Filtrado Seguro

Explotación



Descubrimiento de columnas usando ORDER BY secuencial en un buscador de empleados, culminando en un UNION para extraer documentos confidenciales.

El Parche



Sustituir la concatenación de SQL puro por el método seguro del ORM, garantizando que nunca se concatene el input.

Reto 5: Blind Boolean-Based y la Automatización

Explotación



```
x' OR (SELECT substr(valor,1,1)
FROM secretos)='A' --
```

Extracción de data carácter a carácter basándose únicamente en respuestas lógicas de la aplicación.

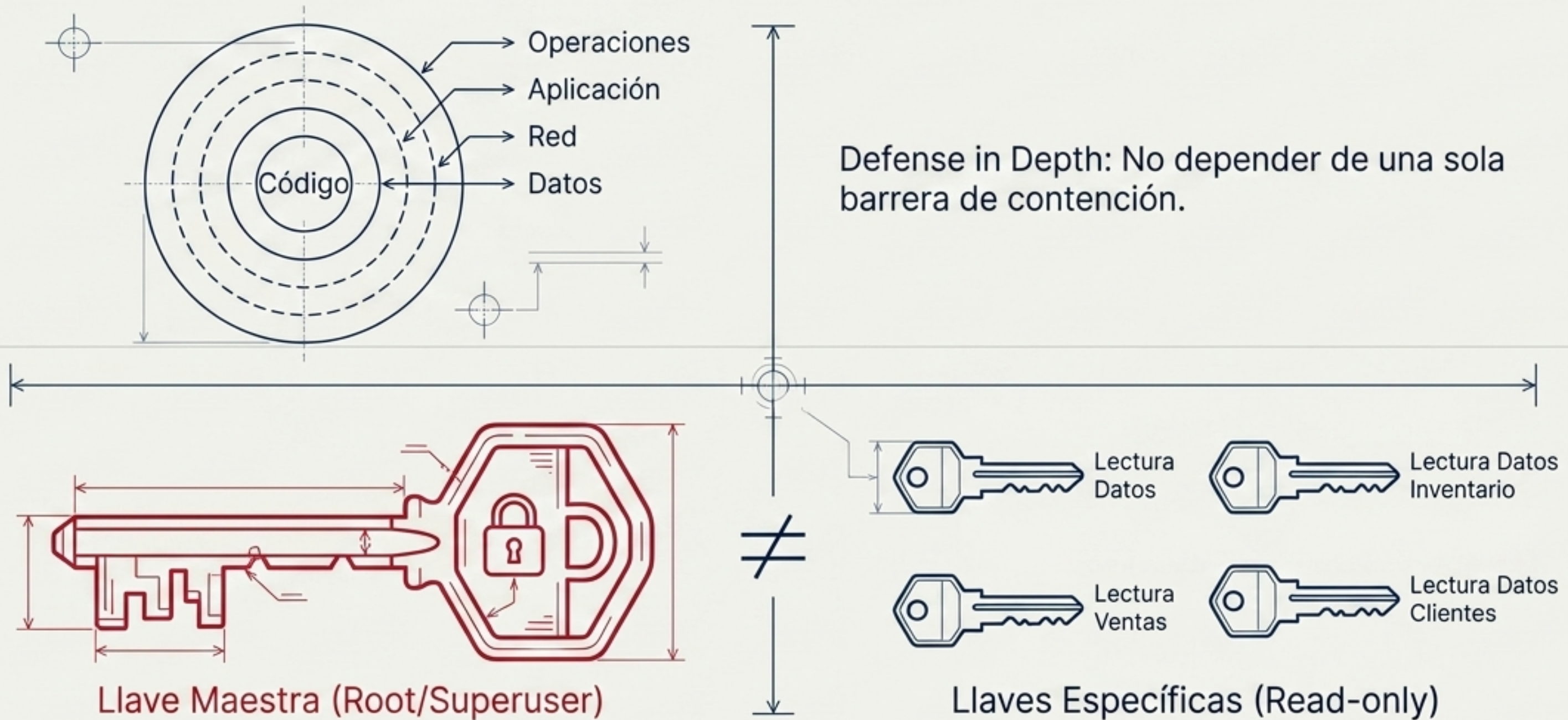
Automatización

```
import requests
url = "http://target.com/vulnerable"
chars = "abcdefghijklmnopqrstuvwxyz0123456789"
data = ""
for i in range(1, 20):
    for char in chars:
        payload = f' AND (SELECT substr(password,{i},1)
        FROM users WHERE username='admin')='{char}' --"
        # ... requests.get(url + payload) ...
```

```
$ sqlmap -u "http://target.com/vulnerable?id=1" --dbs
[INFO] testing connection to the target URL
[INFO] checking if the target is vulnerable
[INFO] GET parameter 'id' is 'AND boolean-based blind
- WHERE or HAVING clause' injectable
```

Uso de scripts en Python y la responsabilidad ética al emplear herramientas automatizadas como sqlmap.

Más allá del código: Defensa en Profundidad



Principio de Mínimo Privilegio: Si la parametrización falla mañana, ¿qué puede hacer la cuenta de base de datos? Nunca operar con privilegios de superusuario si el servicio solo requiere leer datos.

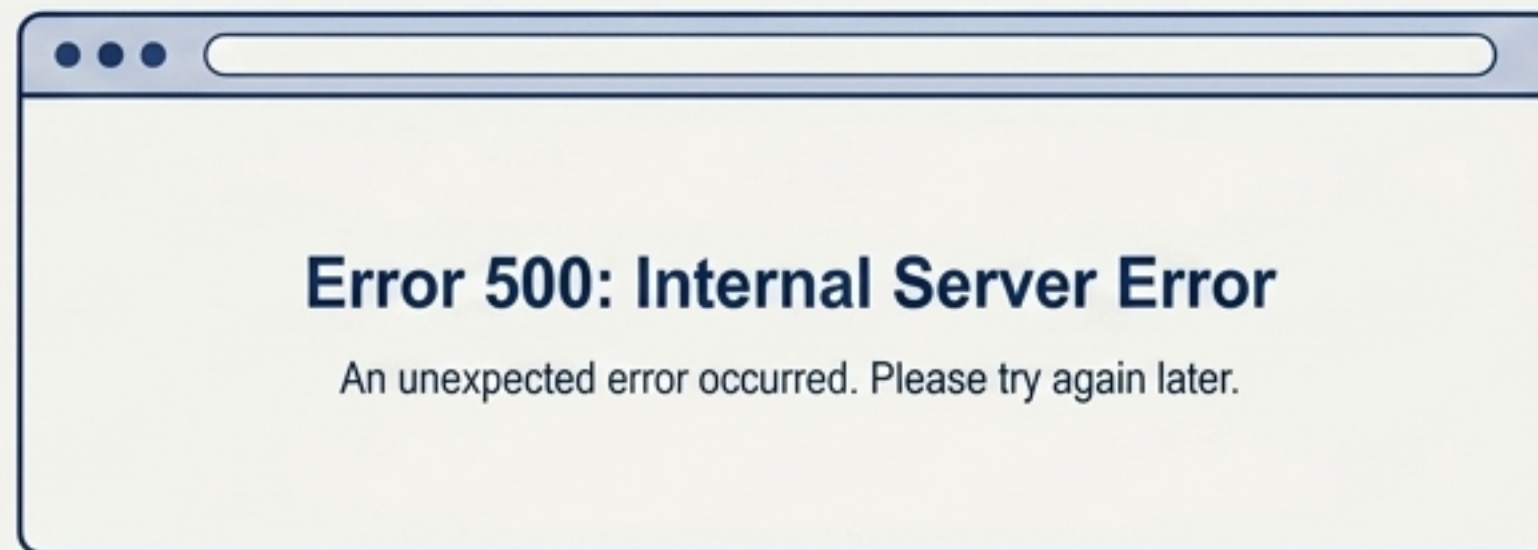
Fallas Silenciosas: El principio de Fail Secure

Vulnerable (Incorrecto)

```
java.sql.SQLException: Connection refused. Check your database username 'admin' and password for 'prod_db_v2'.  
    at com.example.db.ConnectionManager.connect(ConnectionManager.java:45)  
    at com.example.app.UserController.getUserDetails(UserController.java:120)  
Caused by: org.postgresql.util.PSQLException: FATAL: password authentication failed for user "admin"  
Database Name: prod_db_v2; Table: users_table; Engine: PostgreSQL 12.1
```

Los errores detallados (stack traces) son inteligencia gratuita para un atacante.

Seguro (Correcto)



Logs Protegidos
(Interno)

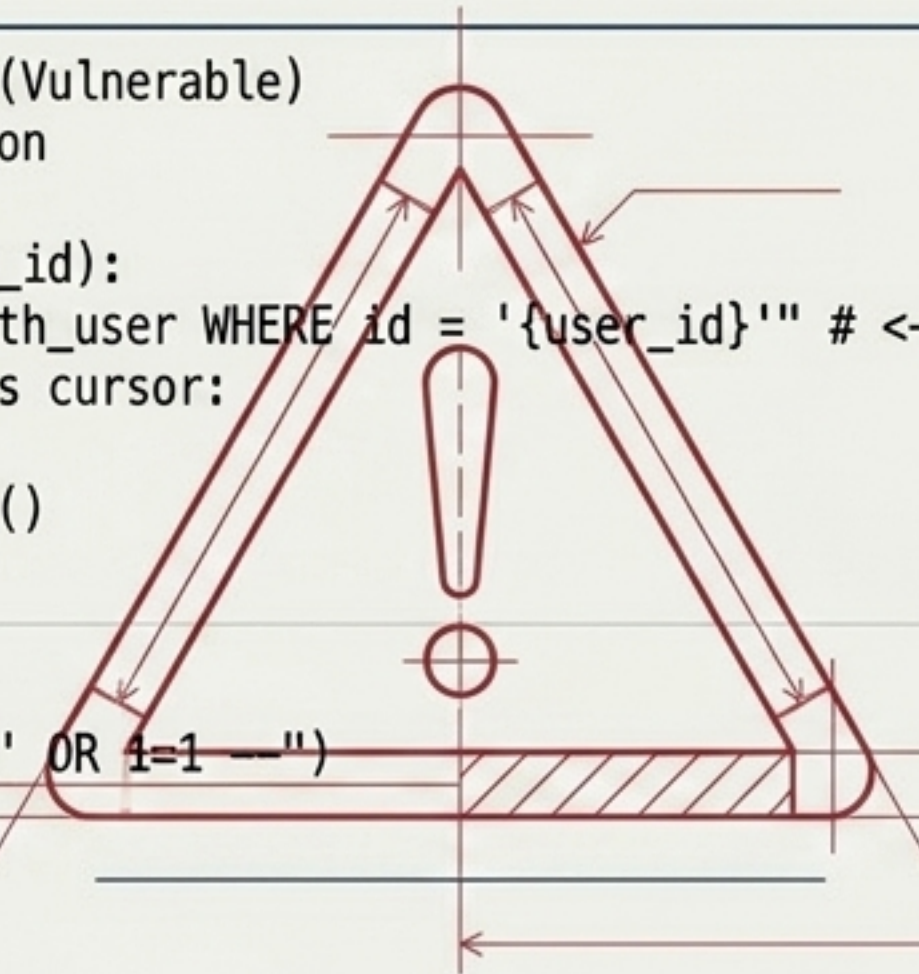
Un sistema Fail Secure mantiene condiciones seguras cuando algo se rompe. El error real se envía a logs protegidos, mientras que el usuario final recibe una respuesta genérica.

El Enemigo Interno: Reintroducción de Vulnerabilidades

```
# Django ORM - Uso Incorrecto (Vulnerable)
from django.db import connection
```

```
def get_user_by_id_unsafe(user_id):
    query = f"SELECT * FROM auth_user WHERE id = '{user_id}'" # <-- Concatenación de cadenas crudas!
    with connection.cursor() as cursor:
        cursor.execute(query)
        user = cursor.fetchone()
    return user
```

```
# Intento de explotación:
# get_user_by_id_unsafe("admin' OR 1=1 --")
```



Falsas sensaciones de seguridad

Un ORM o un Stored Procedure es seguro por defecto, pero si el desarrollador inyecta SQL dinámico internamente mediante métodos crudos para ahorrar tiempo, el escudo desaparece por completo.

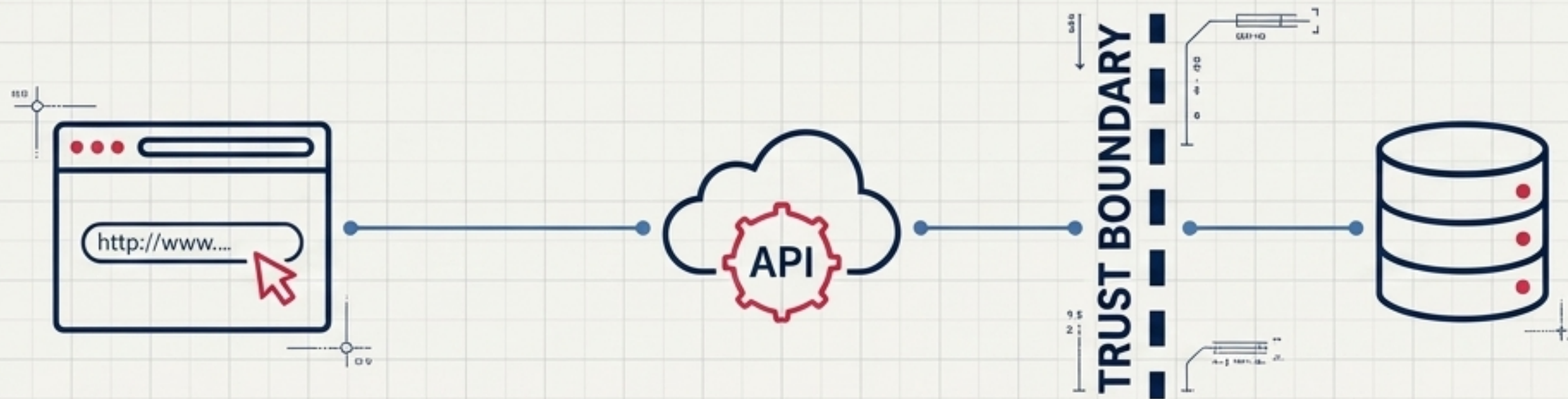
La regla de oro

La regla de oro de no concatenar aplica de manera absoluta en todas las capas de la arquitectura, sin importar qué herramienta moderna envuelva a la base de datos.

Arquitectura Temprana: Shift Left y Trust Boundaries



Shift Left: Integrar la seguridad (SAST/DAST/Bandit) tempranamente.
Secure by Design & Secure by Default.



Tratar cada cruce de datos como hostil. Todo dato debe validarse, **autorizarse y parametrizarse** al cruzar la frontera de confianza.

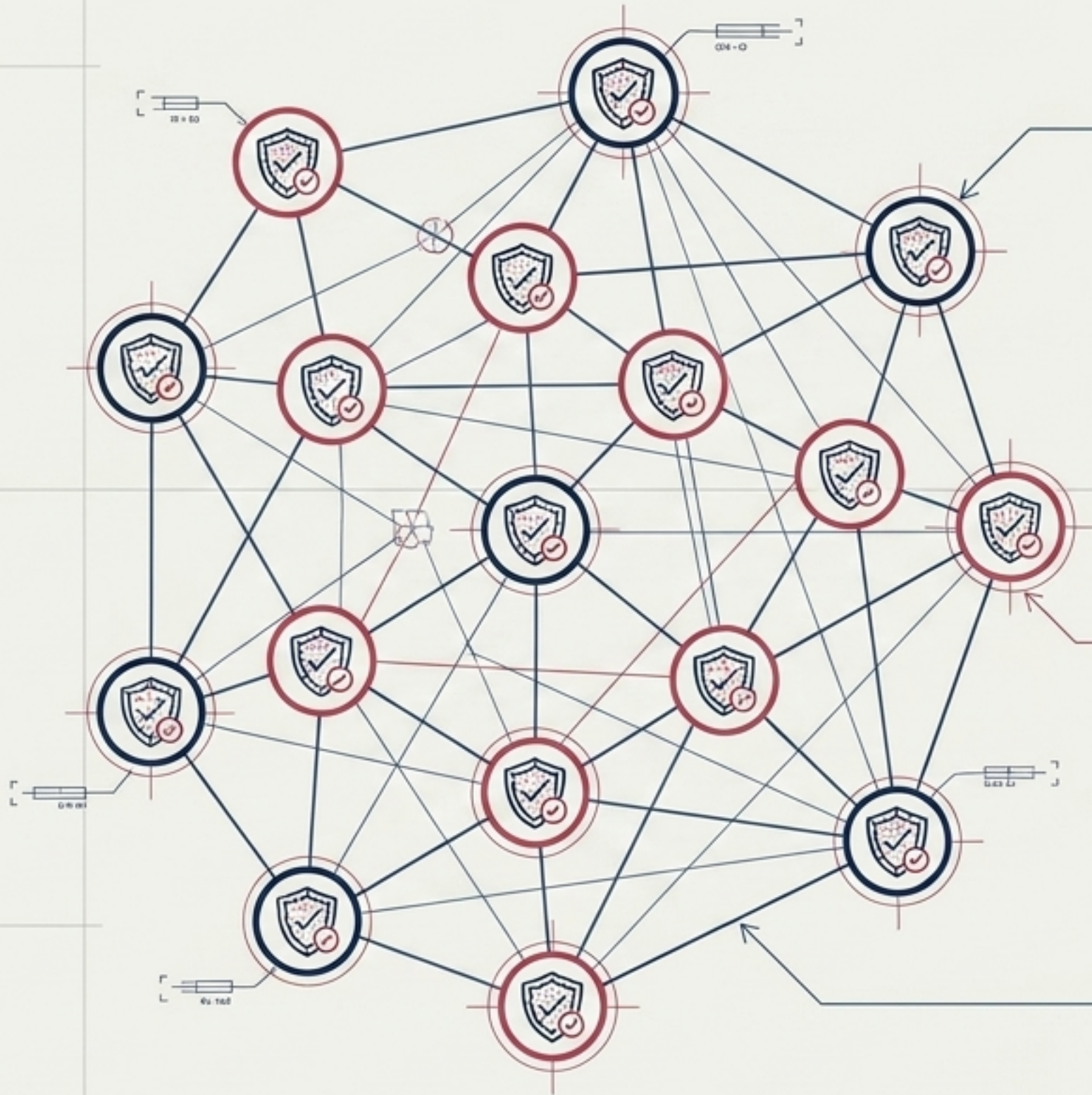
Cambio de Paradigma: Zero Trust y Assume Breach

**Nunca confíes,
siempre verifica.**

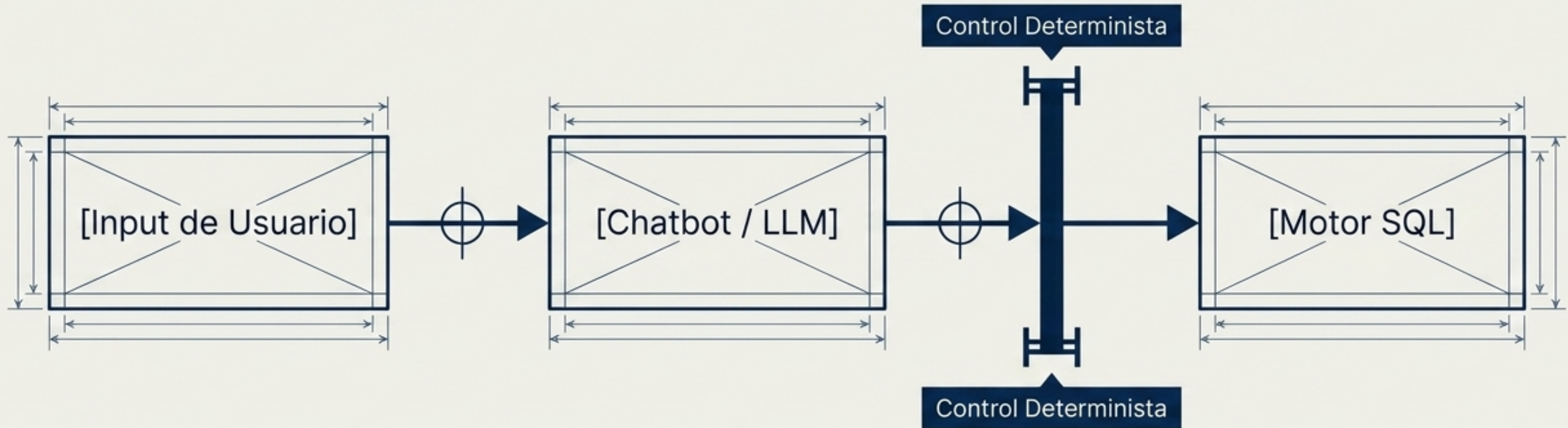
Zero Trust. Estar dentro de la red interna ya no otorga confianza implícita a ningún componente.

**Asume que estás
comprometido.**

Assume Breach. Diseñar la arquitectura asumiendo que un nodo ya ha caído. Si la API web colapsa, la base de datos debe sobrevivir gracias a sus propios controles.



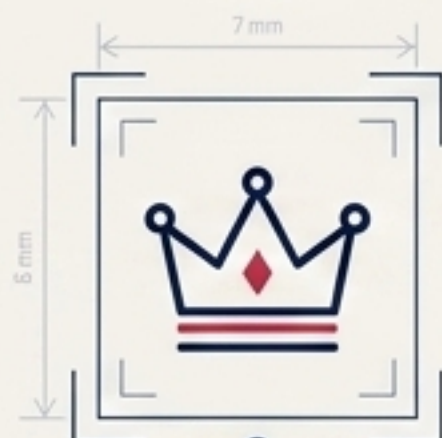
Nuevas Fronteras: SQL Injection en la Era de la IA



Los modelos de lenguaje no pueden ser la frontera de seguridad. La IA genera intenciones y código de manera probabilística y es **susceptible a inyecciones de prompt.**

Una arquitectura segura requiere políticas externas y controles deterministas estrictos. La aplicación debe traducir las salidas de la IA a operaciones fijas y parametrizadas.

El Sustrato Operativo: Conceptos clave de Linux



Root / Sudo

El máximo nivel de compromiso de privilegios en el sistema.



Procesos y Daemons

Cómo se ejecutan los servicios en el servidor (ej. systemd).



Sockets

Canales de comunicación de red a nivel operativo.



Contenedores

Aislamiento del host. Por qué escapar de un contenedor Docker tras vulnerar la BD es la meta del atacante.

La inyección vive en la web, pero el impacto destructivo reside en los permisos del sistema operativo subyacente.

Realidad Profesional: Legalidad y Preguntas Frecuentes

Q: "¿Es esto legal?"

A: "Sí, exclusivamente en entornos autorizados (CTF). Sin permiso escrito, es un delito penado bajo la Ley 21.459 en Chile."

Q: "¿Basta con instalar un WAF?"

A: "Un Web Application Firewall ayuda mitigando ataques conocidos, pero jamás reemplaza la obligación de parchar el código raíz."

Q: "¿Basta con 'encriptar' las contraseñas?"

A: "No. Deben procesarse mediante algoritmos Hash irreversibles con Salt (como PBKDF2 o Bcrypt), no con encriptación bidireccional."

Q: "¿El ORM me protege del ataque CSRF?"

A: "No, son familias de ataques distintos. CSRF fuerza acciones autenticadas desde fuera y requiere protección en la capa middleware, no en la base de datos."

Los datos nunca deben convertirse en instrucciones.

La seguridad no se logra imaginando todas las entradas maliciosas,
sino diseñando sistemas donde la estructura y el usuario jamás se mezclen.